

## **TD7 : les curseurs**

### **Sommaire**

|                                           |   |
|-------------------------------------------|---|
| <a href="#"><u>Introduction</u></a> ..... | 1 |
| <a href="#"><u>Question :1</u></a> .....  | 1 |
| <a href="#"><u>Question :2</u></a> .....  | 1 |
| <a href="#"><u>Question :3</u></a> .....  | 2 |
| <a href="#"><u>Question :4</u></a> .....  | 2 |
| <a href="#"><u>Question :5</u></a> .....  | 2 |
| <a href="#"><u>Question :6</u></a> .....  | 3 |
| <a href="#"><u>Question :7</u></a> .....  | 3 |
| <a href="#"><u>Conclusion</u></a> .....   | 3 |

### **Introduction**

Dans le cadre de ce TP, j'ai étudié le fonctionnement des curseurs en PL/SQL à travers une démarche expérimentale basée sur le cycle prédire  $\rightarrow$  tester  $\rightarrow$  observer  $\rightarrow$  conclure. Mon objectif était de comprendre les différences fondamentales entre les curseurs implicites et explicites, de maîtriser le cycle de vie d'un curseur (déclaration, ouverture, lecture, fermeture) ainsi que ses attributs (%ISOPEN, %FOUND, %NOTFOUND, %ROWCOUNT). J'ai également expérimenté les différentes structures de boucles, le passage de paramètres et la gestion des verrous avec la clause FOR UPDATE.

**(j'ai fait le tp sur oracle en ligne : <https://freesql.com/>)**

## Question :1

J'ai créé deux fonctions stockées : get\_nb\_factures qui utilise COUNT(\*) pour compter les factures d'un client, et get\_chiffre\_affaire qui calcule la somme des montants avec SUM(montant) tout en utilisant NVL pour gérer les clients sans facture.

## Question :2

J'ai écrit un bloc anonyme qui appelle mes deux fonctions en leur passant l'argument 1, puis j'ai affiché le nombre de factures et le chiffre d'affaires du client 1 avec DBMS\_OUTPUT.PUT\_LINE.\_

## Question :3

J'ai utilisé un curseur FOR r\_client IN (SELECT client\_id FROM clients) pour parcourir tous les clients un par un. À chaque itération, j'appelle les deux fonctions et j'affiche le résultat au format demandé avec une ligne de séparation.

## Question :4

J'ai englobé la logique de la question 3 dans une procédure nommée afficher\_etat\_clients, que j'ai ensuite exécutée dans un bloc PL/SQL pour vérifier qu'elle affiche bien l'état de l'ensemble des clients.

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows the schema structure with tables CLIENTS and FACTURES.
- SQL Worksheet:** Contains the following SQL code:
 

```

      1 CREATE TABLE CLIENTS (
      2   client_id NUMBER(10,0) PRIMARY KEY,
      3   nom VARCHAR2(50))
      4 /
      5
      6 CREATE TABLE FACTURES (
      7   facture_id NUMBER(10,0) PRIMARY KEY,
      8   client_id NUMBER(10,0) REFERENCES CLIENTS(client_id),
      9   montant NUMBER(10,2))
      10 /
      11
      12 CREATE OR REPLACE FUNCTION get_nb_factures(p_client_id NUMBER)
      13 RETURN NUMBER
      14 IS
      15   nb_factures NUMBER;
      16 BEGIN
      17   nb_factures := COUNT(*)
      18     FROM FACTURES
      19     WHERE client_id = p_client_id;
      20   RETURN nb_factures;
      21 END;
      22
      23 CREATE OR REPLACE FUNCTION get_chiffre_affaire(p_client_id NUMBER)
      24 RETURN NUMBER
      25 IS
      26   chiffre_affaire NUMBER;
      27 BEGIN
      28   chiffre_affaire := SUM(NVL(montant, 0))
      29     FROM FACTURES
      30     WHERE client_id = p_client_id;
      31   RETURN chiffre_affaire;
      32 END;
      33
      34 CREATE OR REPLACE PROCEDURE afficher_etat_clients
      35 IS
      36   CURSOR c_client IS
      37     SELECT client_id
      38     FROM CLIENTS;
      39 BEGIN
      40   FOR r_client IN c_client
      41   LOOP
      42     DBMS_OUTPUT.PUT_LINE('Client: ' || r_client.client_id);
      43     DBMS_OUTPUT.PUT_LINE('Nb factures: ' || get_nb_factures(r_client.client_id));
      44     DBMS_OUTPUT.PUT_LINE('Chiffre d'affaire: ' || get_chiffre_affaire(r_client.client_id));
      45     DBMS_OUTPUT.PUT_LINE('---');
      46   END LOOP;
      47 END;
      48
      49 BEGIN
      50   afficher_etat_clients;
      51 END;
```
- Query results:** Shows the output of the procedure execution:
 

```

      Procedure afficher_etat_clients completed
      Elapsed: 00:00:00.012

      SQL> /
      Client 1
      2 / 312.25
      Client 2
      1 / 312.34
```
- Table Structure (FACTURES):**

| Column | Column Name | Data Type    | Nullable | Data Default | Column ID | Comments |
|--------|-------------|--------------|----------|--------------|-----------|----------|
| 1      | FACTURE_ID  | NUMBER       | No       | (null)       | 1         | (null)   |
| 2      | CLIENT_ID   | NUMBER       | Yes      | (null)       | 2         | (null)   |
| 3      | MONTANT     | NUMBER(10,2) | Yes      | (null)       | 3         | (null)   |

The screenshot shows the FreeSQL web interface. On the left, the 'Navigator' pane shows the 'CLIENTS' table selected. The main area displays the table structure for 'CLIENTS' with two columns: 'CLIENT\_ID' (NUMBER) and 'NOM' (VARCHAR2(50 BYTE)). Below the table structure, the 'Query result' pane shows the output of a query, displaying two rows of data for 'Client 1' and 'Client 2'.

| Column | COLUMN_NAME | DATA_TYPE         | NULLABLE | DATA_DEFAULT | COLUMN_ID | COMMENTS |
|--------|-------------|-------------------|----------|--------------|-----------|----------|
| 1      | CLIENT_ID   | NUMBER            | No       | (null)       | 1         | (null)   |
| 2      | NOM         | VARCHAR2(50 BYTE) | Yes      | (null)       | 2         | (null)   |

```

SQL> BEGIN
  afficher_etat_clients;
END;

Client 1
2 / 152.23
-----
Client 2
1 / 123.34
-----

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.003
  
```

```

Procedure AFFICHER_ETAT_CLIENTS compiled

Elapsed: 00:00:00.012

SQL> BEGIN
  afficher_etat_clients;
END;

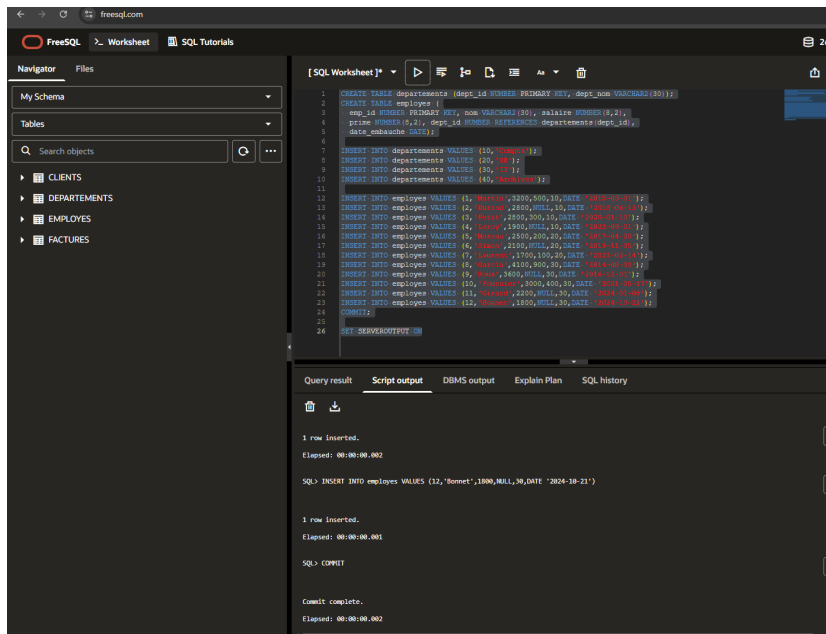
Client 1
2 / 152.23
-----
Client 2
1 / 123.34
-----

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.003
  
```

## Étape 0 : Préparation

Après avoir exécuté le script :



```

1 CREATE TABLE departements (dept_id NUMBER(4) PRIMARY KEY, dept_nom VARCHAR2(30));
2 CREATE TABLE employees (emp_id NUMBER(4) PRIMARY KEY, nom VARCHAR2(30), salaire NUMBER(8,2),
3 prime NUMBER(4,2), dept_id NUMBER(4) REFERENCES departements(dept_id),
4 date_embauche DATE);
5
6
7 INSERT INTO departements VALUES (10,'VENDEURS');
8 INSERT INTO departements VALUES (20,'MKT');
9 INSERT INTO departements VALUES (30,'FIN');
10 INSERT INTO departements VALUES (40,'ACCOMMOD');
11
12 INSERT INTO employees VALUES (1,'MARTIN',2000,500,10,DATE '2018-04-01');
13 INSERT INTO employees VALUES (2,'BENSON',2800,NULL,10,DATE '2018-04-01');
14 INSERT INTO employees VALUES (3,'JONES',2800,500,10,DATE '2018-04-01');
15 INSERT INTO employees VALUES (4,'DEWAY',1800,NULL,10,DATE '2019-09-01');
16 INSERT INTO employees VALUES (5,'BRYANT',2500,200,20,DATE '2017-04-01');
17 INSERT INTO employees VALUES (6,'CLARK',2100,NULL,20,DATE '2017-11-01');
18 INSERT INTO employees VALUES (7,'SCOTT',1700,100,20,DATE '2018-04-01');
19 INSERT INTO employees VALUES (8,'MILLER',1700,50,20,DATE '2018-06-01');
20 INSERT INTO employees VALUES (9,'SMITH',3600,NULL,30,DATE '2018-04-01');
21 INSERT INTO employees VALUES (10,'FORD',3000,400,30,DATE '2018-04-01');
22 INSERT INTO employees VALUES (11,'COOPER',2500,NULL,30,DATE '2018-01-01');
23 INSERT INTO employees VALUES (12,'WARD',1800,NULL,30,DATE '2018-02-01');
24 COMMIT;
25
26 SET SERVEROUTPUT ON
  
```

Query result Script output DBMS output Explain Plan SQL history

1 row inserted.  
Elapsed: 00:00:00.002

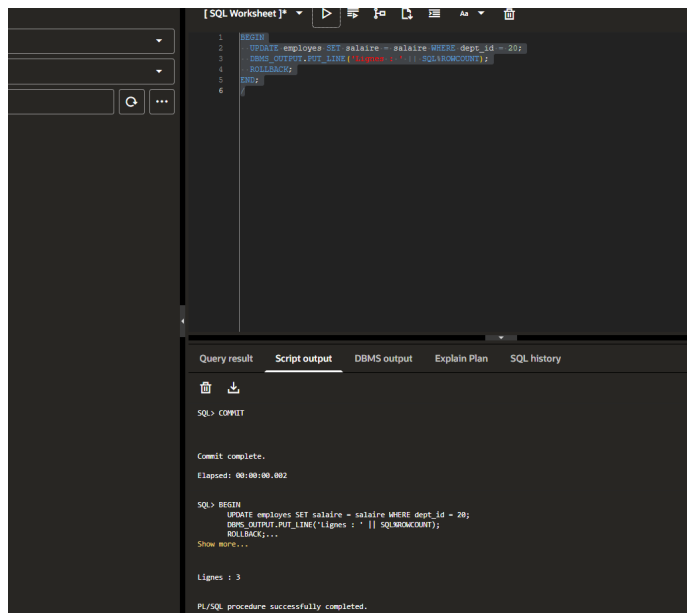
SQL> INSERT INTO employees VALUES (12,'WARD',1800,NULL,30,DATE '2018-02-01')

1 row inserted.  
Elapsed: 00:00:00.001

SQL> COMMIT

Commit complete.  
Elapsed: 00:00:00.002

## Exercice 1 : Le curseur que vous n'avez pas déclaré



```

1 BEGIN
2 UPDATE employees SET salaire = salaire WHERE dept_id = 20;
3 DBMS_OUTPUT.PUT_LINE ('Lignes : ' || SQL%ROWCOUNT);
4 ROLLBACK;
5 END;
6
  
```

Query result Script output DBMS output Explain Plan SQL history

SQL> COMMIT

Commit complete.  
Elapsed: 00:00:00.002

SQL> BEGIN  
UPDATE employees SET salaire = salaire WHERE dept\_id = 20;  
DBMS\_OUTPUT.PUT\_LINE('Lignes : ' || SQL%ROWCOUNT);  
ROLLBACK;

Show more...

Lignes : 3

PL/SQL procedure successfully completed.

Test avec dept\_id = 99.

```
SQL> BEGIN
      UPDATE employees SET salaire = salaire WHERE dept_id = 99;
      DBMS_OUTPUT.PUT_LINE('Lignes : ' || SQL%ROWCOUNT);
      ROLLBACK;...
Show more...

Lignes : 0

PL/SQL procedure successfully completed.
```

## conclusion

Différence de comportement entre UPDATE et SELECT INTO (0 ligne) : Un ordre DML (UPDATE, DELETE) s'exécute toujours sans erreur même s'il ne touche aucune ligne, et met simplement SQL%ROWCOUNT à 0. En revanche, un SELECT INTO exige de trouver au moins une ligne, sinon il lève l'exception NO\_DATA\_FOUND.

- Pourquoi SELECT INTO pose problème si plusieurs lignes sont renvoyées : La variable réceptrice (v\_nom) est une variable scalaire qui ne peut contenir qu'une seule valeur. Si la requête renvoie \$N > 1\$ lignes, PL/SQL déclenche l'erreur TOO\_MANY\_ROWS.

## Exercice 2 : Construire un curseur explicite pas à pas

J'observe que les attributs %FOUND, %NOTFOUND et %ROWCOUNT nécessitent un FETCH pour renvoyer une valeur. Juste après le OPEN, ils valent NULL ou 0. Si le curseur est fermé ou non encore ouvert, toute tentative d'accès à ces trois attributs déclenche l'erreur ORA-01001: invalid cursor. Seul l'attribut %ISOPEN est toujours consultable et permet de vérifier l'état du curseur sans faire planter le programme.

| Étape                      | %ISOPEN | %FOUND                     | %NOTFOUND                  | %ROWCOUNT                  |
|----------------------------|---------|----------------------------|----------------------------|----------------------------|
| <b>Avant OPEN</b>          | FALSE   | NULL                       | NULL                       | 0                          |
| <b>Après OPEN</b>          | TRUE    | NULL                       | NULL                       | 0                          |
| <b>Après 1er<br/>FETCH</b> | TRUE    | TRUE                       | FALSE                      | 1                          |
| <b>Après CLOSE</b>         | FALSE   | <i>Exception ORA-01001</i> | <i>Exception ORA-01001</i> | <i>Exception ORA-01001</i> |

The screenshot displays an SQL Worksheet interface. The top pane contains a PL/SQL script with the following logic:

```

6 BEGIN
12 DBMS_OUTPUT.PUT_LINE('Rowcount : 0');
13
14 -- 2. Après OPEN
15 OPEN c_emp;
16 DBMS_OUTPUT.PUT_LINE('--- Après OPEN ---');
17 DBMS_OUTPUT.PUT_LINE('%ISOPEN : ' || CASE WHEN c_emp%ISOPEN THEN 'TRUE' ELSE 'FALSE' END);
18 DBMS_OUTPUT.PUT_LINE('%FOUND : ' || CASE WHEN c_emp%FOUND THEN 'TRUE' WHEN c_emp%FOUND IS NULL THEN 'NULL' ELSE 'FALSE' END);
19 DBMS_OUTPUT.PUT_LINE('%NOTFOUND : ' || CASE WHEN c_emp%NOTFOUND THEN 'TRUE' WHEN c_emp%NOTFOUND IS NULL THEN 'NULL' ELSE 'FALSE' END);
20 DBMS_OUTPUT.PUT_LINE('Rowcount : ' || c_emp%ROWCOUNT);
21
22 -- 3. Après 1er FETCH
23 FETCH c_emp INTO v_nom, v_salaire;
24 DBMS_OUTPUT.PUT_LINE('--- Après 1er FETCH ---');
25 DBMS_OUTPUT.PUT_LINE('Lu : ' || v_nom || ' (' || v_salaire || ' €)');
26 DBMS_OUTPUT.PUT_LINE('%ISOPEN : ' || CASE WHEN c_emp%ISOPEN THEN 'TRUE' ELSE 'FALSE' END);
27 DBMS_OUTPUT.PUT_LINE('%FOUND : ' || CASE WHEN c_emp%FOUND THEN 'TRUE' WHEN c_emp%FOUND IS NULL THEN 'NULL' ELSE 'FALSE' END);
28 DBMS_OUTPUT.PUT_LINE('%NOTFOUND : ' || CASE WHEN c_emp%NOTFOUND THEN 'TRUE' WHEN c_emp%NOTFOUND IS NULL THEN 'NULL' ELSE 'FALSE' END);
29 DBMS_OUTPUT.PUT_LINE('Rowcount : ' || c_emp%ROWCOUNT);
30
31 -- 4. Après CLOSE
32 CLOSE c_emp;

```

The bottom pane shows the 'Script output' tab with the following execution results:

```

SQL> DECLARE
CURSOR c_emp IS
SELECT nom, salaire FROM employees WHERE dept_id = 30;
v_nom employees.nom%TYPE;...
Show more...

--- Avant OPEN ---
%ISOPEN : FALSE
%FOUND : NULL (non utilisable avant OPEN)
%NOTFOUND : NULL (non utilisable avant OPEN)
%ROWCOUNT : 0
--- Après OPEN ---
%ISOPEN : TRUE
%FOUND : NULL
%NOTFOUND : NULL
%ROWCOUNT : 0
--- Après 1er FETCH ---
Lu : Garcia (4100 €)
%ISOPEN : TRUE
%FOUND : TRUE
%NOTFOUND : FALSE
%ROWCOUNT : 1
--- Après CLOSE ---
%ISOPEN : FALSE

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.012

```

At the bottom of the interface, there are links for 'Privacy Rights', 'Delete Your FreeSQL Account', and 'Préférences en matière de cookies', along with a page number '26/100' and a copyright notice 'Copyright © 2015-2026 Oracle'.

## Exercice 3 : Casser le curseur volontairement

### Conclusion

Ce TP m'a permis d'acquérir une bonne compréhension pratique de la manipulation des données en PL/SQL. J'ai pu constater que le choix de la structure de boucle est déterminant pour éviter des erreurs courantes : la boucle FOR rec IN curseur s'est révélée être la plus fiable, car elle automatise la gestion du curseur et limite les risques de fuites de ressources. De plus, l'expérimentation du verrouillage avec FOR UPDATE et WHERE CURRENT OF m'a montré comment garantir la cohérence des données lors d'accès concurrents. Ces notions me permettent désormais de savoir exactement quand privilégier un curseur explicite face à un simple SELECT INTO.